

## UGV navigation with dynamic obstacles

Objective: Obstacles may appear or move during navigation. The UGV must detect them and calculate a new route.

```
import random
import heapq
import matplotlib.pyplot as plt

SIZE = 20

START = (0, 0)
GOAL = (19, 19)

def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar(grid, start, goal):
    pq = [(0, start)]
    cost = {start: 0}
    parent = {}

    while pq:
        _, current = heapq.heappop(pq)

        if current == goal:
            path = []

            while current in parent:
                path.append(current)
                current = parent[current]

            path.append(start)
            path.reverse()

            return path

        x, y = current

        for nx, ny in [
            (x + 1, y),
            (x - 1, y),
            (x, y + 1),
            (x, y - 1)
        ]:
            if 0 <= nx < SIZE and 0 <= ny < SIZE:
                if grid[nx][ny] == 1:
                    continue

                new_cost = cost[current] + 1

                if new_cost < cost.get((nx, ny), float('inf')):
                    cost[(nx, ny)] = new_cost
                    parent[(nx, ny)] = current

                    priority = new_cost + heuristic((nx, ny), goal)

                    heapq.heappush(
                        pq,
                        (priority, (nx, ny))
                    )

    return None

random.seed(10)

grid = [[0] * SIZE for _ in range(SIZE)]

for i in range(SIZE):
    for j in range(SIZE):
```

```

        if (i, j) not in [START, GOAL]:
            if random.random() < 0.15:
                grid[i][j] = 1

current = START
actual_path = [START]
replans = 0
max_steps = SIZE * SIZE
for step in range(max_steps):
    if current == GOAL:
        break

    path = astar(grid, current, GOAL)

    if path is None:
        print("No possible route")
        break

    next_position = path[1]

    if random.random() < 0.20:
        if next_position != GOAL:
            grid[next_position[0]][next_position[1]] = 1

            print("Dynamic obstacle detected at:", next_position)

            replans += 1
            continue

    current = next_position

    actual_path.append(current)

    if random.random() < 0.10:
        ox = random.randint(0, SIZE - 1)
        oy = random.randint(0, SIZE - 1)

        if (ox, oy) not in [current, GOAL, START]:
            grid[ox][oy] = 1

            print("New obstacle:", (ox, oy))

            replans += 1

print("\nDynamic UGV Navigation Results")

print("Start:", START)
print("Goal:", GOAL)
print("Reached Goal:", current == GOAL)
print("Actual Distance:", len(actual_path) - 1)
print("Number of Replanning Operations:", replans)

plt.figure(figsize=(8, 8))

plt.imshow(grid, cmap="Greys", origin="upper")

x = [p[1] for p in actual_path]
y = [p[0] for p in actual_path]

plt.plot(x, y, color="blue", linewidth=2, label="UGV Movement")

plt.scatter(START[1], START[0], color="green", s=100, label="Start")
plt.scatter(GOAL[1], GOAL[0], color="red", s=100, label="Goal")

plt.title("UGV Navigation with Dynamic Obstacles")
plt.legend()
plt.show()

```

```
Dynamic obstacle detected at: (0, 3)
New obstacle: (19, 4)
Dynamic obstacle detected at: (3, 4)
Dynamic obstacle detected at: (4, 8)
New obstacle: (2, 11)
Dynamic obstacle detected at: (7, 15)
Dynamic obstacle detected at: (8, 14)
Dynamic obstacle detected at: (12, 15)
Dynamic obstacle detected at: (12, 17)
New obstacle: (6, 0)
New obstacle: (13, 11)
Dynamic obstacle detected at: (18, 16)
Dynamic obstacle detected at: (16, 14)
Dynamic obstacle detected at: (15, 15)
Dynamic obstacle detected at: (14, 15)
Dynamic obstacle detected at: (12, 14)
Dynamic obstacle detected at: (11, 13)
New obstacle: (13, 9)
Dynamic obstacle detected at: (16, 12)
Dynamic obstacle detected at: (15, 13)
No possible route
```

Dynamic UGV Navigation Results  
Start: (0, 0)  
Goal: (19, 19)  
Reached Goal: False  
Actual Distance: 63  
Number of Replanning Operations: 20

